DuitLater · Multi-Cloud Setup Guide

Prime · for Mung (Backend Lead)

2026-04-25

Multi-Cloud Setup & HA Failover Guide

DuitLater · TNG FINHACK 2026 · Financial Inclusion Track Audience: Mung (Backend / Foundation-Keeper) — primary implementer **Companion to:** `infra/RELEASE.md` (single-EC2 baseline runbook) **Status:** v1.0 · 2026-04-25

Ringkasan Eksekutif

Guide ni tunjuk macam mana setup **3 EC2 server** sebagai HA cluster dengan **Cloudflare** sebagai gate masuk + load balancer + auto-failover, plus integrasi dengan **Alibaba Cloud Function Compute** untuk AI workloads. Semua orkestrasi dirancang untuk **active-passive failover** — Server 1 selalu jadi primary, Server 2 dan 3 backup secara automatik bila primary down.

Topology summary: - 3 EC2 di AWS ap-southeast-1 (semua run full DuitLater stack) - Cloudflare Load Balancer (Pro plan) handle DNS + health monitoring + auto-failover - Postgres streaming replication antara 3 server (Server 1 primary, Server 2 + 3 read-only replicas) - Alibaba Cloud Function Compute host AI inference (Penasihat suggester · NADI summary) - Cross-cloud backup: Postgres → AWS S3 → mirror ke Alibaba OSS

Detection time untuk failover: ~90 saat (3 missed health checks at 30s interval). **Manual DB promotion time:** ~10 saat (one command via SSH). **Total downtime untuk writes** semasa failover: ~2 minit. Reads continue tanpa gangguan.

Daftar Kandungan

---

1. Konsep Asas

1.1 High Availability (HA)

HA = sistem yang masih boleh berfungsi **walaupun ada bahagian yang rosak**. Daripada 1 server (single point of failure), kita gunakan beberapa server. Bila satu down, yang lain ambil alih. Pengguna tak sedar apa-apa terjadi.

1.2 Failover

Failover = proses tukar dari server primary ke server backup. Boleh **automatik** (Cloudflare detect server down dan auto-route ke backup) atau **manual** (sysadmin SSH dan promote replica jadi primary).

Untuk DuitLater: - Web traffic failover = **automatik** (Cloudflare detect ~90s) - Database write failover = **manual** (Mung run satu command, ~10s)

1.3 Active-Passive vs Active-Active

Mode	Penjelasan	Sesuai bila
Active-Passive	1 server serve traffic, lain-lain standby (idle). Kalau active down, salah satu standby ambil alih.	Setup ringkas. DB write split mudah.
Active-Active	Semua server serve traffic concurrently. Load balanced.	Setup kompleks. DB write split kena multi-master atau read-replica + smart routing.

DuitLater pakai Active-Passive — paling sesuai untuk hackathon scope dengan single-primary Postgres.

1.4 Postgres Streaming Replication

Mechanism untuk sync data antara Postgres servers:

1. Server primary tulis data → simpan dalam **Write-Ahead Log (WAL)** (binary log file)
2. WAL stream via TCP ke replica servers
3. Replica apply WAL → data muncul di replica DB
4. Replica = read-only (tak boleh terima write)

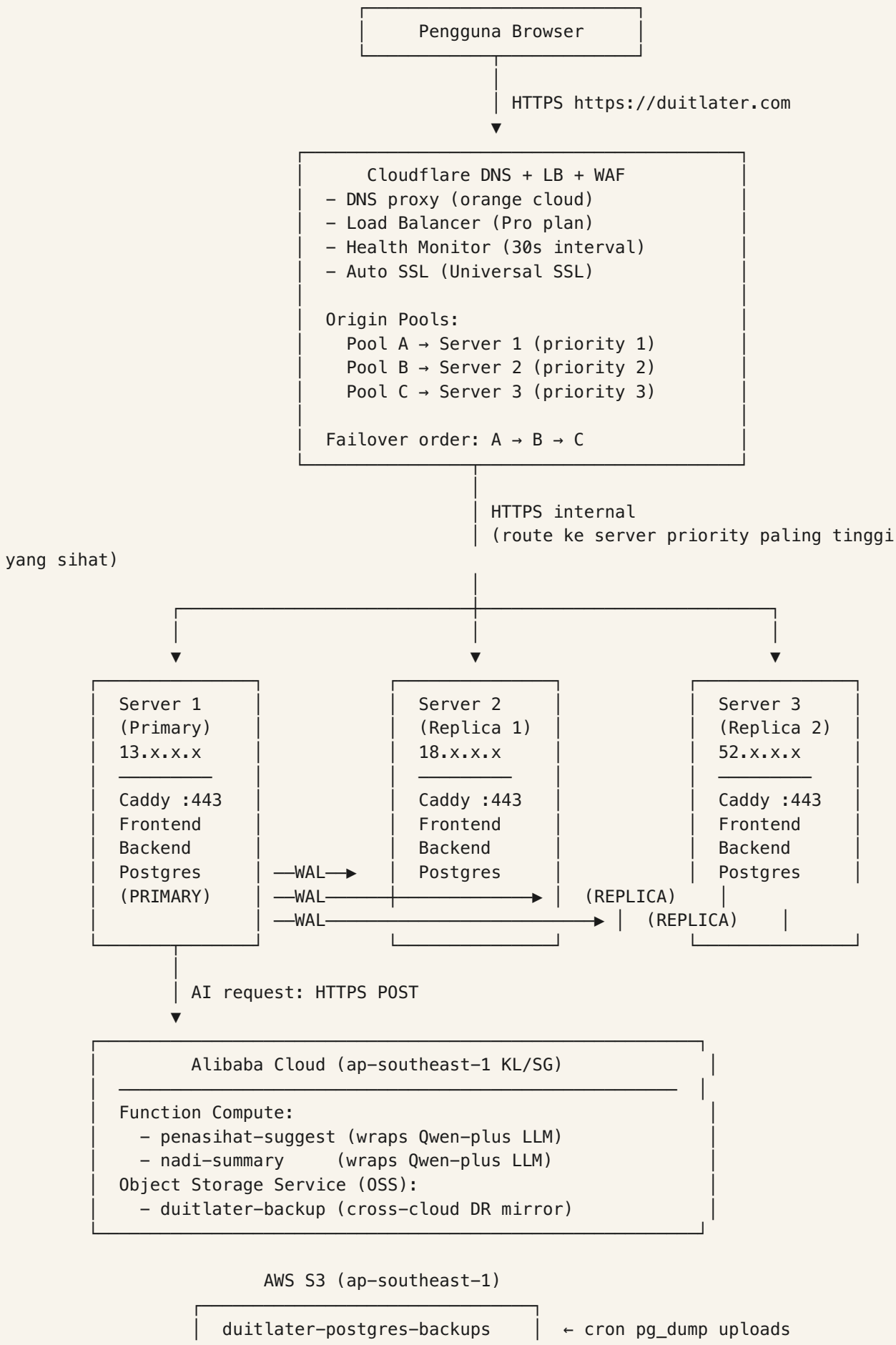
5. Lag biasa: ~10-100 milliseconds (async mode)

Async vs Sync mode: - **Async** — primary tak tunggu replica acknowledge sebelum commit. Faster writes, tiny risk of data loss kalau primary crash sebelum WAL stream sampai (~last 100ms writes mungkin hilang). - **Sync** — primary tunggu sekurang-kurangnya 1 replica acknowledge. Slower writes, zero data loss.

DuitLater pakai async — untuk hackathon, kelajuan write lagi penting dari guarantee zero loss. Acceptable trade-off.

---

2. Topology Penuh


Pengguna Browser
HTTPS https://duitlater.com
Cloudflare DNS + LB + WAF
- DNS proxy (orange cloud)
- Load Balancer (Pro plan)
- Health Monitor (30s interval)
- Auto SSL (Universal SSL)
Origin Pools:
Pool A → Server 1 (priority 1)
Pool B → Server 2 (priority 2)
Pool C → Server 3 (priority 3)
Failover order: A → B → C
HTTPS internal
(route ke server priority paling tinggi
yang sihat)
Server 1
(Primary)
13.x.x.x
Caddy :443
Frontend
Backend
Postgres
(PRIMARY)
Server 2
(Replica 1)
18.x.x.x
Caddy :443
Frontend
Backend
Postgres
Server 3
(Replica 2)
52.x.x.x
Caddy :443
Frontend
Backend
Postgres
WAL
WAL
WAL
(REPLICA)
(REPLICA)
AI request: HTTPS POST
Alibaba Cloud (ap-southeast-1 KL/SG)
Function Compute:
- penasihat-suggest (wraps Qwen-plus LLM)
- nadi-summary (wraps Qwen-plus LLM)
Object Storage Service (OSS):
- duitlater-backup (cross-cloud DR mirror)
AWS S3 (ap-southeast-1)
duitlater-postgres-backups
← cron pg_dump uploads

```
                    │  duitlater-assets                │  ← general static assets
                    └──────────────────────────────────┘
```

2.1 Komponen Utama

Komponen	Fungsi	Lokasi
Cloudflare DNS	Resolve `duitlater.com` ke Cloudflare proxy	Cloud
Cloudflare Load Balancer	Health check + auto-failover routing	Cloud
Server 1 (EC2 t3.medium)	Active web stack + Postgres primary	AWS ap-southeast-1
Server 2 (EC2 t3.medium)	Standby web stack + Postgres replica	AWS ap-southeast-1
Server 3 (EC2 t3.medium)	Standby web stack + Postgres replica	AWS ap-southeast-1
Alibaba Function Compute	Penasihat AI · NADI summary AI	Alibaba Cloud
AWS S3	Postgres backup + static assets	AWS
Alibaba Cloud OSS	Cross-cloud DR backup mirror	Alibaba

2.2 Network Flow Examples

Normal request (semua sihat):

```
Pengguna → Cloudflare → Server 1 (active) → DB write succeeds → response
```

Server 1 down:

```
Pengguna → Cloudflare → Server 2 (took over) → DB read OK · DB write FAILS (read-only)
                                                → 503 returned to user (until manual
promote)
```

Lepas manual promote Server 2:

```
Pengguna → Cloudflare → Server 2 (now primary) → DB writes work → response
```

3. Prasyarat

3.1 Akaun & Akses

Yang Mung perlu	Sebab
AWS account dengan permission EC2 + EIP + S3 + IAM	Provision 3 server + backup bucket
Cloudflare account (Pro plan minimum, $20/month)	Load Balancer feature kena Pro+
Alibaba Cloud account dengan permission FC + OSS + DashScope	Multi-cloud AI workloads + DR mirror
Domain (e.g., `duitlater.com`) di Cloudflare	DNS gateway
GitHub PAT classic dengan `read:packages` scope	Pull pre-built backend image dari GHCR
1 SSH key pair (.pem file)	Connect ke 3 EC2

3.2 Tools Tempatan

```
# Mung install ni di laptop/workstation:
brew install awscli                    # AWS CLI
brew install cloudflared              # Cloudflare CLI (optional)
pip3 install aliyun-cli               # Alibaba CLI
brew install postgresql@17            # Postgres client (psql, pg_dump)
```

3.3 Maklumat yang Mung kena pegang

Sebelum mula, sediakan dalam satu file selamat (e.g., 1Password atau secure notes):

```
=== AWS ===
EC2 Server 1: i-xxxxx · IP 13.x.x.x · SSH key: ~/.ssh/duitlater.pem
EC2 Server 2: i-xxxxx · IP 18.x.x.x · SSH key: ~/.ssh/duitlater.pem
EC2 Server 3: i-xxxxx · IP 52.x.x.x · SSH key: ~/.ssh/duitlater.pem
Region: ap-southeast-1
S3 bucket: duitlater-postgres-backups

=== Cloudflare ===
Domain: duitlater.com
Account ID: xxxxxxxxxxxx
Zone ID: xxxxxxxxxxxx
API Token: cf_xxxxx (Zone:DNS:Edit + Zone:Load Balancers:Edit)

=== Alibaba Cloud ===
Account: xxxxx@xxx.com
Access Key ID: LTAI...
Access Key Secret: xxxxx
Region: ap-southeast-1 (or 3 = KL)
DashScope API Key: sk-xxxxx
FC Service: duitlater-fc
OSS Bucket: duitlater-backup-mirror

=== Postgres ===
Master Password (for replication): generate via `openssl rand -base64 32`
Replication User: replicator
Replication Password: generate via `openssl rand -base64 32`
```

Bahagian A — Setup AWS Side

A.1 Provision 3 EC2

Ikut `infra/RELEASE.md` **Section 2** untuk setiap server. Ulang 3 kali. **Beza** dari single-server runbook:

Setting	Server 1	Server 2	Server 3
Name tag	`duitlater-server-1`	`duitlater-server-2`	`duitlater-server-3`
AMI	Ubuntu 24.04 LTS	sama	sama
Type	t3.medium	t3.medium	t3.medium
Storage	30 GB gp3	30 GB gp3	30 GB gp3
Security Group	`duitlater-sg` (shared)	sama	sama
Key pair	`duitlater.pem` (shared)	sama	sama
Elastic IP	EIP-1	EIP-2	EIP-3

Security Group (`duitlater-sg`) rules:

```
Inbound:
  Port 22  (SSH)            from your IP only
  Port 80  (HTTP)           from 0.0.0.0/0  (Cloudflare proxy will hit this)
  Port 443 (HTTPS)          from 0.0.0.0/0  (Cloudflare proxy)
  Port 5432 (Postgres rep)  from same SG (so 3 servers boleh sync antara satu sama
lain)

Outbound:
  All allowed (default)
```

PENTING: Port 5432 hanya buka antara 3 server (intra-SG). Tak buka ke Internet — Postgres tak boleh expose publicly.

A.2 Install Docker Stack on Each Server

Untuk setiap server (Server 1, 2, 3), SSH masuk dan jalan:

```
# Per `infra/RELEASE.md` Section 3
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-ip>

# Update + install dependencies
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git ca-certificates gnupg lsb-release

# Install Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/
        apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
        compose-plugin
sudo usermod -aG docker ubuntu

# Logout + login balik so docker group apply
exit
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-ip>

# Login ke GHCR (untuk pull backend image)
echo "<github-pat>" | docker login ghcr.io -u <github-username> --password-stdin

# Clone repo
cd ~
git clone https://github.com/Zen0space/R2-D2-Finhack.git duitlater
cd duitlater
git checkout main  # atau dev untuk dev stack

# Install pnpm (untuk DB migrations)
curl -fsSL https://get.pnpm.io/install.sh | sh -
export PATH="$HOME/.local/share/pnpm:$PATH"
echo 'export PATH="$HOME/.local/share/pnpm:$PATH"' >> ~/.bashrc

# Pull pre-built backend image
docker pull ghcr.io/zen0space/duitlater-backend:latest
```

A.3 Setup Postgres Primary (Server 1)

Server 1 = primary. Bina volume yang persist + configure WAL streaming.

```
# SSH ke Server 1
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-1-ip>
cd ~/duitlater

# Buat .env files
cat > packages/backend/.env.prod <<EOF
NODE_ENV=production
PORT=4000
DATABASE_URL=postgresql://duitlater:<generate-password>@postgres:5432/duitlater?
        schema=public
ALIBABA_FUNCTION_COMPUTE_URL=  # akan set lepas Bahagian B
ALIBABA_FUNCTION_COMPUTE_URL_NADI=
ALIBABA_FUNCTION_COMPUTE_KEY=
ANTHROPIC_API_KEY=             # fallback
TNG_API_BASE=https://sandbox.tngwallet.com.my
LOG_LEVEL=info
EOF

cat > infra/.env.prod <<EOF
POSTGRES_USER=duitlater
POSTGRES_PASSWORD=<generate-password>
POSTGRES_DB=duitlater
POSTGRES_PORT=5432
EOF

# Start prod stack pertama kali (untuk inisial DB)
cd infra
docker compose -f docker-compose.prod.yml -p prod up -d postgres
sleep 10  # Beri Postgres masa untuk init

# Run Prisma migration (setup schema)
cd ..
pnpm --filter db install
DATABASE_URL="postgresql://duitlater:<password>@localhost:5432/duitlater?
        schema=public" \
  pnpm --filter db migrate
```

Configure WAL streaming pada Server 1's Postgres

Tukar Postgres config supaya boleh stream WAL ke replicas:

```
# Edit Postgres config dalam container
docker exec -it duitlater-prod-postgres bash

# Inside container:
cat >> /var/lib/postgresql/data/postgresql.conf <<EOF

# === Replication settings ===
wal_level = replica
max_wal_senders = 5
wal_keep_size = 1GB
hot_standby = on
synchronous_commit = on  # tukar ke 'off' kalau nak async (faster)
EOF

# Add replication permissions
cat >> /var/lib/postgresql/data/pg_hba.conf <<EOF

# === Replication clients (Server 2 + 3) ===
host  replication  replicator  <server-2-private-ip>/32  md5
host  replication  replicator  <server-3-private-ip>/32  md5
EOF

# Create replication user
psql -U duitlater -d duitlater <<EOF
CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD '<rep-password>';
EOF

exit  # Keluar dari container

# Restart Postgres untuk apply changes
docker restart duitlater-prod-postgres
sleep 10

# Verify replication user
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT usename, userepl FROM pg_user WHERE usename='replicator';"
```

A.4 Setup Postgres Replicas (Server 2 + 3)

Buat untuk **Server 2** dan **Server 3** (sama steps).

```
# SSH ke Server 2 (atau 3)
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-2-ip>
cd ~/duitlater

# Setup .env files (sama dengan Server 1)
# (copy dari Server 1 dengan scp atau retype)

# Start Postgres container WITHOUT initial data (kita nak base backup dari Server 1)
cd infra
docker compose -f docker-compose.prod.yml -p prod up -d postgres
sleep 5

# Stop Postgres dulu (kena ganti data dengan base backup)
docker stop duitlater-prod-postgres

# Wipe data volume
docker volume rm duitlater_prod_postgres_data 2>/dev/null || true
docker volume create duitlater_prod_postgres_data

# Run pg_basebackup dari Server 1 (primary) — copy semua data + WAL state
docker run --rm \
  -v duitlater_prod_postgres_data:/var/lib/postgresql/data \
  -e PGPASSWORD='<replicator-password>' \
  postgres:17-alpine \
  pg_basebackup -h <server-1-private-ip> -p 5432 -U replicator \
                -D /var/lib/postgresql/data \
                -X stream -P -R

# Note: -R flag auto-create standby.signal + primary_conninfo dalam
        postgresql.auto.conf
# Bermakna replica auto-detect dirinya sebagai standby, dan stream WAL dari Server 1

# Start Postgres balik (sekarang dalam standby mode)
docker compose -f docker-compose.prod.yml -p prod up -d postgres

# Verify replica is in recovery (read-only) mode
sleep 10
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT pg_is_in_recovery();"
# Expected: t  (true = recovery mode = replica)
```

Ulang same steps untuk **Server 3**.

A.5 Verify Replication

Pada **Server 1** (primary), check status:

```
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-1-ip>
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater <<EOF
SELECT
  client_addr,
  state,
  sent_lsn,
  write_lsn,
  flush_lsn,
  replay_lsn,
  sync_state
FROM pg_stat_replication;
EOF
```

Expected output: 2 baris (Server 2 + Server 3), `state = streaming`, `sync_state = async`.

Test write/read:

```
# Pada Server 1 (primary), insert
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "INSERT INTO \"HealthCheck\" (id, message) VALUES ('test-1', 'replication test');"

# Pada Server 2 (replica), check muncul
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT * FROM \"HealthCheck\" WHERE id='test-1';"
# Expected: 1 row · same data · within ~1 second

# Try write pada Server 2 (kena fail)
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "INSERT INTO \"HealthCheck\" (id, message) VALUES ('fail', 'test');"
# Expected: ERROR: cannot execute INSERT in a read-only transaction
```

Kalau semua ni jalan = replication setup OK.

A.6 Setup Cloudflare DNS + Load Balancer

Step 1: Add domain ke Cloudflare

1. Login ke Cloudflare dashboard
2. Add Site → enter `duitlater.com`
3. Pilih Pro plan ($20/month)
4. Cloudflare bagi 2 nameservers — update kat domain registrar Mung

Step 2: Initial DNS A records (proxied)

Dalam Cloudflare → DNS → Records:

```
Type: A   Name: @       Content: <Server 1 EIP>   Proxy: ON (orange cloud)
Type: A   Name: www     Content: <Server 1 EIP>   Proxy: ON
Type: A   Name: dev     Content: <Server 1 EIP>   Proxy: ON  (optional, untuk dev
stack)
```

Note: kita nanti akan ganti dengan Load Balancer, tapi DNS records ni jadi fallback.

Step 3: Buat Origin Pools

Dalam Cloudflare → Traffic → Load Balancing:

Pool A — Server 1 Primary:

```
Name: duitlater-server-1
Origins:
  - Name: server-1
    Address: <Server 1 EIP>
    Weight: 1
    Enabled: yes
Health monitor: (akan setup di Step 5)
Notification email: ijam@duitlater.com (or team chat)
```

Pool B — Server 2 Standby:

```
Name: duitlater-server-2
Origins:
  - Name: server-2
    Address: <Server 2 EIP>
    Weight: 1
```

Pool C — Server 3 Standby:

```
Name: duitlater-server-3
Origins:
  - Name: server-3
    Address: <Server 3 EIP>
    Weight: 1
```

Step 4: Buat Health Monitor

Cloudflare → Traffic → Load Balancing → Health Monitors:

```
Name: duitlater-health
Type: HTTPS
Method: GET
Path: /api/v1/health
Expected codes: 200
Interval: 30 seconds
Retries: 2
Timeout: 5 seconds
Follow redirects: yes
Allow insecure cert: no
Header: Host: duitlater.com  ← important so Caddy match the right vhost
```

Attach this monitor to all 3 pools.

Step 5: Buat Load Balancer

Cloudflare → Traffic → Load Balancing → Load Balancers → Create:

```
Hostname: duitlater.com
Status: Active

Pool fallback order (FAILOVER MODE):
  1. Pool A (Server 1)   ← primary
  2. Pool B (Server 2)   ← if A unhealthy
  3. Pool C (Server 3)   ← if A + B unhealthy

Steering policy: Off (Failover order)  ← important untuk priority-based
Session affinity: None
Proxy mode: HTTP/HTTPS
```

Save. Cloudflare auto-update DNS records untuk point ke Load Balancer endpoint.

A.7 Health Endpoint + Monitor

Backend kita dah ada `/api/v1/health` endpoint (per `packages/backend/src/index.ts`). Verify dia respond OK pada setiap server:

```
# Test setiap server directly (bypass Cloudflare)
curl -k https://13.x.x.x/api/v1/health    # Server 1
curl -k https://18.x.x.x/api/v1/health    # Server 2
curl -k https://52.x.x.x/api/v1/health    # Server 3

# Expected: { "status": "ok", "env": "production", "timestamp": "..." }
```

Cloudflare akan auto-detect “healthy” status setelah 1 minit (1 successful health check).

A.8 Test Failover

Test 1 — Normal state:

```
curl https://duitlater.com/api/v1/health
# Expected: respond 200 (from Server 1)

# Check Cloudflare LB analytics dashboard
# → all 3 pools = healthy
# → traffic going to Pool A
```

Test 2 — Simulate Server 1 down:

```
# SSH ke Server 1, stop app container
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-1-ip>
docker stop duitlater-prod-app

# Wait 90 saat (3 missed health checks at 30s)
# Then test from outside:
curl https://duitlater.com/api/v1/health
# Expected: respond 200 (from Server 2)
# Check Cloudflare LB dashboard:
# → Pool A unhealthy
# → traffic auto-routed to Pool B
```

Test 3 — Server 1 up balik:

```
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-1-ip>
docker start duitlater-prod-app

# Wait 60 saat (1 successful health check)
curl https://duitlater.com/api/v1/health
# Expected: respond 200 (from Server 1 again)
# → Pool A healthy
# → traffic returns to Pool A
```

Test 4 — Server 1 + 2 down:

```
ssh ubuntu@server-1-ip docker stop duitlater-prod-app
ssh ubuntu@server-2-ip docker stop duitlater-prod-app

# Wait 90s
curl https://duitlater.com/api/v1/health
# Expected: respond 200 (from Server 3)
```

Semua test pass = web tier failover **automatic** working. Database write failover masih **manual** — see Bahagian D.

Bahagian B — Setup Alibaba Cloud Side

B.1 Create Alibaba Cloud Account

1. Sign up di alibabacloud.com
2. Verify identity (Malaysian credit card OK)
3. Get access — boleh ambil masa beberapa jam untuk activation

B.2 Get DashScope API Key (Qwen)

DashScope = Alibaba's LLM service yang host Qwen models.

1. Login ke Alibaba Cloud Console
2. Search "DashScope" → klik "Model Studio"
3. Sidebar → "API Keys" → Create new
4. Copy key — format: `sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx`
5. Simpan secure (akan masuk Function Compute env var nanti)

B.3 Deploy `penasihat-suggest` Function

Step 1: Create FC Service

1. Console → Function Compute → Services → Create
2. Name: `duitlater-fc`
3. Region: `ap-southeast-1` (Singapore) — closest to KL
4. Description: "DuitLater AI workloads — Penasihat suggester + NADI summary"
5. Save

Step 2: Create Function

Inside `duitlater-fc` service:

1. Click "Create Function"
2. **Method:** "Use a built-in runtime"
3. Configuration:

```
Function name: penasihat-suggest
Runtime: Node.js 18
Memory: 512 MB
Timeout: 10 seconds
Triggers: HTTP trigger
    Authentication: anonymous (atau dengan key kalau Mung mahu lock)
    HTTP methods: POST
    Disable web protection: yes (HTTP trigger akan handle)
```

4. Code source: "Upload zip"

Step 3: Prepare zip

Pada laptop Mung:

```
cd /path/to/R2-D2-Finhack/alibaba-function-compute/penasihat-suggest
zip -r penasihat-suggest.zip index.js package.json
ls -la penasihat-suggest.zip  # confirm exists
```

Step 4: Upload + configure

Back to Function Compute Console: 1. Click “Upload zip” → pick `penasihat-suggest.zip` 2. Environment variables: `DASHSCOPE_API_KEY = sk-xxxxxxxxxxxxxxxxxxxxxxxxxx (dari B.2) NODE_ENV = production` 3. Save

Step 5: Get HTTP trigger URL

1. Function detail page → “Triggers” tab
2. Copy “Public URL” — looks like:

   ```
   https://duitlater-fc.fcv3.<region>.fc.aliyuncs.com/2023-03-30/proxy/duitlater-fc/
   penasihat-suggest/
   ```

3. Save URL — akan masuk ke `.env.prod` di Step B.5

Step 6: Test the function

```
curl -X POST "<your-fc-url>" \
  -H "Content-Type: application/json" \
  -d '{
    "context": {
      "poolId": "test-1",
      "combinedCapCents": 200000,
      "statedNeed": "Mesin jahit untuk mula tailoring",
      "statedNeedCategory": "EQUIPMENT",
      "kampungName": "Felda Gedangsa",
      "monthOfYear": 4
    },
    "candidates": [
      {"id": "1", "name_bm": "Mesin jahit Brother", "category": "EQUIPMENT",
        "price_cents": 180000},
      {"id": "2", "name_bm": "Beras 100kg", "category": "GROCERY", "price_cents":
        28000},
      {"id": "3", "name_bm": "Generator 2.5kVA", "category": "EQUIPMENT",
        "price_cents": 120000}
    ]
  }'
```

Expected: JSON response dengan top items dalam BM. Kalau dapat 5xx — check `DASHSCOPE_API_KEY` dah set, dan function logs dalam Cloudflare console.

B.4 Deploy `nadi-summary` Function

Sama dengan penasihat-suggest, tapi: - Function name: `nadi-summary` - Code: take from `alibaba-function-compute/nadi-summary/index.js` (kalau ada · belum ditulis · atau tulis sekarang based on `backend/src/services/nadi-summary.ts`) - Different HTTP trigger URL

Note: Mung boleh skip ni dulu kalau Phase 5b belum priority. Phase 3 (Penasihat suggester) lagi penting.

B.5 Configure Backend `.env.prod` on All 3 EC2

SSH ke setiap server, update `packages/backend/.env.prod`:

```
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-ip>
cd ~/duitlater
nano packages/backend/.env.prod

# Tambah/ubah:
ALIBABA_FUNCTION_COMPUTE_URL=https://duitlater-fc.fcv3.<region>.fc.aliyuncs.com/
        2023-03-30/proxy/duitlater-fc/penasihat-suggest/
ALIBABA_FUNCTION_COMPUTE_URL_NADI=https://duitlater-fc.fcv3.<region>.fc.aliyuncs.com/
        2023-03-30/proxy/duitlater-fc/nadi-summary/
ALIBABA_FUNCTION_COMPUTE_KEY=  # leave empty kalau anonymous trigger
ANTHROPIC_API_KEY=sk-ant-xxxxx  # fallback kalau Alibaba down

# Save
```

Restart backend container untuk apply:

```
cd infra
docker compose -f docker-compose.prod.yml -p prod restart app
```

Test integration dari server (or via Cloudflare):

```
# Trigger Penasihat suggest endpoint
curl -X POST https://duitlater.com/api/v1/penasihat/suggest \
  -H "Content-Type: application/json" \
  -d '{"poolId": "test-pool"}'

# Backend akan call Alibaba FC, return suggestions
# Check response includes provider="alibaba-qwen"
```

Ulang untuk Server 2 + Server 3. Multi-cloud routing live.

Bahagian C — Backup Cross-Cloud

Tujuan: Postgres data backup ke AWS S3 (hourly) + cross-cloud mirror ke Alibaba OSS (daily) untuk DR.

C.1 Postgres pg_dump Cron

Pada **Server 1** (primary saja — replica auto-sync):

```
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-1-ip>
sudo mkdir -p /opt/duitlater/backups
sudo chown ubuntu:ubuntu /opt/duitlater/backups

# Backup script
cat > /opt/duitlater/backup-pg.sh <<'EOF'
#!/bin/bash
set -e
TIMESTAMP=$(date -u +%Y%m%dT%H%M%SZ)
BACKUP_FILE="/opt/duitlater/backups/duitlater-${TIMESTAMP}.sql.gz"

# Dump from running container
docker exec duitlater-prod-postgres \
  pg_dump -U duitlater -d duitlater --format=custom --compress=9 \
  > "${BACKUP_FILE}.tmp"

mv "${BACKUP_FILE}.tmp" "${BACKUP_FILE}"

# Upload to S3
aws s3 cp "${BACKUP_FILE}" \
  "s3://duitlater-postgres-backups/$(date -u +%Y/%m/%d)/$(basename ${BACKUP_FILE})" \
  --storage-class STANDARD_IA

# Keep only last 24 local backups (24 jam x 1 backup per jam)
find /opt/duitlater/backups -name 'duitlater-*.sql.gz' -mtime +1 -delete

echo "Backup OK: ${BACKUP_FILE}"
EOF

chmod +x /opt/duitlater/backup-pg.sh

# Test manual sekali
/opt/duitlater/backup-pg.sh

# Add to cron (every hour)
crontab -e
# Tambah line ni:
# 0 * * * * /opt/duitlater/backup-pg.sh >> /var/log/duitlater-backup.log 2>&1
```

C.2 Upload to AWS S3

Need AWS CLI configured pada Server 1:

```
# Pada Server 1
aws configure
# AWS Access Key ID: <buat IAM user dengan S3 write permission>
# AWS Secret Access Key: <secret>
# Default region: ap-southeast-1
# Default output format: json

# Create bucket kalau belum ada
aws s3 mb s3://duitlater-postgres-backups --region ap-southeast-1

# Set lifecycle policy: delete old backups after 30 days
cat > /tmp/lifecycle.json <<EOF
{
  "Rules": [
    {
      "Id": "delete-after-30d",
      "Status": "Enabled",
      "Prefix": "",
      "Expiration": { "Days": 30 }
    }
  ]
}
EOF
aws s3api put-bucket-lifecycle-configuration \
  --bucket duitlater-postgres-backups \
  --lifecycle-configuration file:///tmp/lifecycle.json
```

C.3 Mirror S3 → Alibaba OSS (Daily)

Cross-cloud DR: kalau seluruh AWS region down, masih ada backup di Alibaba OSS.

Step 1: Create OSS bucket

```
# Alibaba Cloud Console → OSS → Create Bucket
# Name: duitlater-backup-mirror
# Region: ap-southeast-1 (Singapore) atau ap-southeast-3 (KL)
# Storage class: Standard
# Versioning: Enabled (untuk safety)
```

Step 2: Setup mirror script

```
# Pada Server 1
sudo apt install -y python3-pip
pip3 install --user oss2

cat > /opt/duitlater/mirror-to-oss.py <<'PYEOF'
#!/usr/bin/env python3
"""Mirror today's Postgres backups from AWS S3 → Alibaba OSS."""
import os
import sys
import boto3
import oss2
from datetime import datetime, timezone

S3_BUCKET = "duitlater-postgres-backups"
OSS_BUCKET = "duitlater-backup-mirror"
OSS_ENDPOINT = "https://oss-ap-southeast-1.aliyuncs.com"
OSS_KEY = os.environ["ALIBABA_OSS_ACCESS_KEY"]
OSS_SECRET = os.environ["ALIBABA_OSS_SECRET"]

today = datetime.now(timezone.utc).strftime("%Y/%m/%d")
prefix = f"{today}/"

s3 = boto3.client("s3", region_name="ap-southeast-1")
auth = oss2.Auth(OSS_KEY, OSS_SECRET)
oss_bucket = oss2.Bucket(auth, OSS_ENDPOINT, OSS_BUCKET)

# List today's S3 backups
resp = s3.list_objects_v2(Bucket=S3_BUCKET, Prefix=prefix)
contents = resp.get("Contents", [])
print(f"Found {len(contents)} backups in S3 for {today}")

for obj in contents:
    key = obj["Key"]
    if oss_bucket.object_exists(key):
        print(f"  skip (exists): {key}")
        continue

    # Stream copy: S3 → memory → OSS
    body = s3.get_object(Bucket=S3_BUCKET, Key=key)["Body"].read()
    oss_bucket.put_object(key, body)
    print(f"  mirrored: {key} ({len(body)} bytes)")

print("Mirror done.")
PYEOF

chmod +x /opt/duitlater/mirror-to-oss.py

# Configure env vars
cat > /opt/duitlater/.oss-env <<EOF
export ALIBABA_OSS_ACCESS_KEY=<your-alibaba-access-key>
export ALIBABA_OSS_SECRET=<your-alibaba-secret>
EOF
chmod 600 /opt/duitlater/.oss-env

# Test manual
source /opt/duitlater/.oss-env && /opt/duitlater/mirror-to-oss.py
```

```
# Add to cron (daily at 02:00 UTC = 10:00 MYT)
crontab -e
# Tambah:
# 0 2 * * * source /opt/duitlater/.oss-env && /opt/duitlater/mirror-to-oss.py >> /var/
        log/duitlater-mirror.log 2>&1
```

C.4 Restore Drill (Test pemulihan)

Sebulan sekali, test restore (sangat penting — backup yang tak pernah test = backup yang tak boleh dipakai):

```
# Download a backup dari S3
aws s3 cp s3://duitlater-postgres-backups/2026/04/25/duitlater-20260425T060000Z.sql.gz
        \
  /tmp/restore-test.sql.gz

# Spin up a fresh test Postgres container
docker run -d --name pg-restore-test \
  -e POSTGRES_PASSWORD=test \
  -p 5433:5432 \
  postgres:17-alpine
sleep 10

# Restore
gunzip -c /tmp/restore-test.sql.gz | \
  docker exec -i pg-restore-test pg_restore -U postgres -d postgres --create

# Verify data
docker exec pg-restore-test psql -U postgres -c "\\dt"

# Cleanup
docker stop pg-restore-test && docker rm pg-restore-test
rm /tmp/restore-test.sql.gz

echo "Restore drill OK"
```

Bahagian D — Failover Playbook

D.1 Web Failover (Auto)

Triggered: Cloudflare detect 3 missed health checks (90 saat). **Action:** Cloudflare auto-route traffic ke pool berikutnya dalam priority order. **Monitoring:** Cloudflare dashboard → Traffic → Load Balancing → Analytics.

Tiada manual action diperlukan untuk web tier. Pengguna mungkin nampak ~90 saat slow/error, lepas tu auto-recover.

D.2 Database Promotion (Manual)

Bila perlu: Server 1 (primary) down + Mung nak Server 2 jadi primary supaya writes work balik.

Pre-conditions: - Server 1 confirmed down (tak respond SSH or shutdown EC2) - Cloudflare dah route traffic ke Server 2 (auto) - Backend pada Server 2 cuba write → fail dengan “read-only transaction” error

Steps:

```
# 1. SSH to Server 2
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-2-ip>

# 2. Promote Postgres replica to primary
docker exec duitlater-prod-postgres \
  pg_ctl promote -D /var/lib/postgresql/data

# Verify promotion
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT pg_is_in_recovery();"
# Expected: f  (false = no longer in recovery = is primary)

# 3. Restart backend container untuk reset connection pool
cd ~/duitlater/infra
docker compose -f docker-compose.prod.yml -p prod restart app

# 4. Test write
curl -X POST https://duitlater.com/api/v1/some-write-endpoint \
  -H "Content-Type: application/json" \
  -d '{"test":"after-promotion"}'
# Expected: 200 success (write went to Server 2's promoted DB)
```

Total time: ~10 saat command + ~30 saat backend restart = 40 saat. Plus 90 saat Cloudflare detection = ~2 minit total downtime untuk writes.

PENTING: After promotion, Server 1 (when up balik) **TIDAK BOLEH directly serve traffic dengan data lama dia**. Server 1 mesti re-sync dari Server 2 sebagai replica. See Section D.3.

D.3 Recovery — Server 1 Up Balik

Selepas Mung dah promote Server 2 jadi primary, kalau Server 1 up balik:

Problem: Server 1’s Postgres masih ingat dia primary (data lama). Server 2 sekarang primary (data baru). Cloudflare akan auto-route traffic balik ke Server 1 sebab priority 1, tapi Server 1’s data **stale** — risiko data corruption.

Solution: Re-sync Server 1 sebagai replica from Server 2.

```
# 1. SSH to Server 1
ssh -i ~/.ssh/duitlater.pem ubuntu@<server-1-ip>

# 2. STOP backend immediately (jangan biar dia serve traffic dengan data lama)
cd ~/duitlater/infra
docker compose -f docker-compose.prod.yml -p prod stop app

# 3. Stop Postgres container
docker stop duitlater-prod-postgres

# 4. Wipe old data volume
docker volume rm duitlater_prod_postgres_data
docker volume create duitlater_prod_postgres_data

# 5. Re-base from Server 2 (sekarang primary baru)
docker run --rm \
  -v duitlater_prod_postgres_data:/var/lib/postgresql/data \
  -e PGPASSWORD='<replicator-password>' \
  postgres:17-alpine \
  pg_basebackup -h <server-2-private-ip> -p 5432 -U replicator \
                -D /var/lib/postgresql/data \
                -X stream -P -R

# 6. Start Postgres balik (sekarang dalam standby mode, replicating dari Server 2)
docker compose -f docker-compose.prod.yml -p prod up -d postgres
sleep 10

# 7. Verify replica state
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT pg_is_in_recovery();"
# Expected: t (true · is replica)

# 8. Start backend
docker compose -f docker-compose.prod.yml -p prod up -d app
```

Sekarang topology jadi: - **Server 2 = primary** (yang ambik alih masa Server 1 down) - **Server 1 = replica** (re-synced from Server 2) - **Server 3 = replica** (still replicating from old primary lineage; might need re-sync too)

Perhati Server 3 — kalau dia masih replica dari Server 1 (yang dah crash), dia mungkin stuck atau diverge. Re-base Server 3 from Server 2 sama macam Step 4-7 di atas.

Cloudflare priority: traffic balik ke Server 1 (priority 1, healthy). Tapi Server 1's Postgres = replica sekarang, so writes still go through Server 2. Backend connection string pada Server 1 kena point ke Server 2's Postgres untuk writes — atau gunakan a connection pooler like PgBouncer with read/write split.

Untuk hackathon scope: simplest, just leave Server 2 sebagai primary sampai event habis. Don't fail back. Kalau Server 1 up balik tapi Server 2 dah amik tempat = leave alone.

D.4 Failback (Optional · Post-hackathon)

Kalau Mung nak revert kembali (Server 1 jadi primary lagi):

```
# 1. Pause writes (announce maintenance window)
# 2. Wait for Server 1 (current replica) to fully catch up dengan Server 2 (primary)
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT pg_last_wal_receive_lsn() = pg_last_wal_replay_lsn();"
# Expected: t (caught up)

# 3. Stop Server 2's Postgres (current primary)
ssh ubuntu@server-2-ip docker stop duitlater-prod-postgres

# 4. Promote Server 1's Postgres
ssh ubuntu@server-1-ip docker exec duitlater-prod-postgres \
  pg_ctl promote -D /var/lib/postgresql/data

# 5. Re-base Server 2 + Server 3 as replicas from Server 1
# (sama macam D.3 step 4-7)

# 6. Restart backends
# 7. Resume traffic
```

Recommended: Don't failback during hackathon. Stable > clean.

Bahagian E — Verification Checklist

Mung tick satu-satu sebelum demo:

Pre-deploy

- [ ] 3 EC2 (Server 1, 2, 3) provisioned, t3.medium, ap-southeast-1
- [ ] Security group allows port 22 (your IP), 80/443 (anywhere), 5432 (intra-SG)
- [ ] Each EC2 has Elastic IP attached
- [ ] Domain `duitlater.com` di Cloudflare (Pro plan), nameservers updated
- [ ] Cloudflare Pro plan paid + activated
- [ ] AWS S3 bucket `duitlater-postgres-backups` created with 30d lifecycle
- [ ] Alibaba Cloud account active
- [ ] Alibaba OSS bucket `duitlater-backup-mirror` created
- [ ] DashScope API key obtained

Stack deploy

- [ ] Docker installed on all 3 EC2
- [ ] GHCR login successful on all 3
- [ ] Backend image `duitlater-backend:latest` pulled on all 3
- [ ] `.env.prod` files configured on all 3 (with same secrets)
- [ ] Postgres container up on Server 1 with WAL streaming config
- [ ] Postgres replication user `replicator` created
- [ ] `pg_hba.conf` allows replication from Server 2 + 3 IPs

Replication

- [ ] Server 2 ran `pg_basebackup` from Server 1
- [ ] Server 3 ran `pg_basebackup` from Server 1
- [ ] Both replicas in standby mode (`pg_is_in_recovery() = true`)
- [ ] `SELECT * FROM pg_stat_replication` on Server 1 shows 2 active streams
- [ ] Test INSERT on Server 1 visible on Server 2 + 3 within 5 seconds
- [ ] Write attempt on Server 2 returns “read-only transaction” error

Cloudflare LB

- [ ] 3 origin pools created (server-1, server-2, server-3)
- [ ] Health monitor active (HTTPS GET /api/v1/health · 30s · 2 retries)
- [ ] Load Balancer created with failover order [Pool A, B, C]
- [ ] Steering policy: “Off (Failover order)”
- [ ] DNS resolves `duitlater.com` to Cloudflare proxy
- [ ]

All 3 pools showing “healthy” on dashboard

Multi-cloud (Alibaba)

- [ ] FC service `duitlater-fc` created in ap-southeast-1
- [ ] Function `penasihat-suggest` deployed (Node.js 18 · 512 MB · 10s timeout)
- [ ] HTTP trigger created (POST · anonymous)
- [ ] `DASHSCOPE_API_KEY` set in function env
- [ ] Test curl returns valid suggestions
- [ ] Backend `.env.prod` updated with `ALIBABA_FUNCTION_COMPUTE_URL` on all 3 EC2
- [ ] Backend restart applied
- [ ] `provider="alibaba-qwen"` in response when calling `/api/v1/penasihat/suggest`

Backup

- [ ] `pg_dump` cron running on Server 1 (every hour)
- [ ] AWS CLI configured on Server 1 (`aws s3 ls` works)
- [ ] First backup uploaded to S3 successful
- [ ] OSS mirror script tested manually
- [ ] OSS daily mirror cron added (02:00 UTC)
- [ ] Restore drill completed once (test backup file restorable)

Failover tests

- [ ] Test 1 — Stop Server 1 app, verify Cloudflare routes to Server 2 within 90s
- [ ] Test 2 — Restart Server 1 app, verify Cloudflare returns to Server 1 within 60s
- [ ] Test 3 — DB promotion playbook run end-to-end on Server 2
- [ ]

- [ ] Test 4 — Re-sync Server 1 as replica successful
- [ ] Test 5 — Stop Server 1 + Server 2, verify routes to Server 3

Demo readiness

- [ ] All 3 servers showing “healthy” on Cloudflare LB dashboard
- [ ] `https://duitlater.com/api/v1/health` returns 200 from any browser
- [ ] Pool formation flow works end-to-end
- [ ] Penasihat suggestion API returns BM-first results within 6s
- [ ] Failover playbook printed/saved offline (in case Mung phone tak ada signal)
- [ ] AWS Console + Cloudflare dashboard tabs ready on Mung’s laptop

---

Bahagian F — Troubleshooting

F.1 Cloudflare LB shows pool unhealthy but server is up

Symptom: Cloudflare dashboard shows Pool A unhealthy, but `curl <server-1-ip>/api/v1/health` works.

Cause: Cloudflare hits server via internal Cloudflare IPs (different from your IP). Server’s security group might not allow.

Fix: - Ensure security group allows ALL of Cloudflare’s IPs on 80/443 - Cloudflare IP list: https://www.cloudflare.com/ips-v4/ - Or set Origin Pool’s “host header” to `duitlater.com` so Caddy match the right vhost

F.2 Replica falls behind — data not syncing

Symptom: `pg_stat_replication` shows large `lag` value.

Causes: - Network bandwidth between servers saturated - Replica server CPU bound on apply WAL - WAL files getting deleted on primary before replica can stream

Fix: - Increase `wal_keep_size` on primary (default 1GB → try 4GB) - Use replication slots: ```sql – On primary SELECT pg_create_physical_replication_slot(‘replica_slot_2’);

– On replica (in postgresql.auto.conf) primary_slot_name = ‘replica_slot_2’ ``` - Replication slots prevent primary from deleting WAL until replica has consumed it

F.3 Promotion fails — replica still in recovery mode

Symptom: `pg_ctl promote` runs without error, but `pg_is_in_recovery()` still returns `t`.

Cause: Promotion takes a few seconds to complete.

Fix: Wait 10-30 seconds, then check again. If still recovery, check Postgres logs:

```
docker logs duitlater-prod-postgres --tail 100
```

F.4 Backend can't connect to Postgres after promotion

Symptom: Backend logs "connection refused" after promoting Server 2.

Cause: Connection pool still has connections to old primary (Server 1, now down).

Fix: Restart backend container:

```
cd ~/duitlater/infra
docker compose -f docker-compose.prod.yml -p prod restart app
```

F.5 Alibaba FC return 5xx or timeout

Symptom: Penasihat call returns slow/empty, backend logs show `Alibaba Function Compute failed`.

Cause possibilities: - DashScope API rate limit hit (Qwen rate-limited) - Alibaba FC region down - DashScope API key expired

Fix: - Check Alibaba Cloud status page - Backend auto-falls-back to Anthropic Claude per the routing logic in `services/penasihat.ts` - Verify Claude API key in `.env.prod` - Re-test FC endpoint directly with curl

F.6 Cross-cloud OSS mirror fails

Symptom: Mirror script log shows AccessDenied or timeout.

Cause: Alibaba OSS access key might not have write permission, or wrong endpoint region.

Fix: - Re-check `ALIBABA_OSS_ACCESS_KEY` and `ALIBABA_OSS_SECRET` env vars - Verify OSS bucket policy allows writes from your IAM user - Try `oss2.Bucket(...).list_objects()` first to test read permission

F.7 Cloudflare TLS handshake fails

Symptom: Cloudflare dashboard shows "Origin not reachable", Caddy logs show TLS errors.

Cause: Cloudflare ? Caddy SSL not configured.

Fix: - In Cloudflare, set SSL/TLS encryption mode to "Full" or "Full (strict)" - Caddy auto-generates Let's Encrypt cert on first request - For "Full (strict)", Caddy must serve a valid CA-signed cert (Let's Encrypt is fine)

Lampiran — Quick Reference

Common Commands

```
# === Per-server health check ===
curl -k https://13.x.x.x/api/v1/health    # Server 1
curl -k https://18.x.x.x/api/v1/health    # Server 2
curl -k https://52.x.x.x/api/v1/health    # Server 3

# === Cloudflare LB endpoint ===
curl https://duitlater.com/api/v1/health

# === Postgres replication status (run on primary) ===
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT * FROM pg_stat_replication;"

# === Promote replica ke primary ===
docker exec duitlater-prod-postgres pg_ctl promote -D /var/lib/postgresql/data

# === Check is replica? ===
docker exec duitlater-prod-postgres psql -U duitlater -d duitlater \
  -c "SELECT pg_is_in_recovery();"
# t = replica, f = primary

# === Restart backend ===
docker compose -f infra/docker-compose.prod.yml -p prod restart app

# === Backup manual ===
/opt/duitlater/backup-pg.sh

# === Mirror to OSS manual ===
source /opt/duitlater/.oss-env && /opt/duitlater/mirror-to-oss.py

# === Test Alibaba FC ===
curl -X POST <fc-url> -H "Content-Type: application/json" -d '{...}'
```

Ports

Port	Where	What
22	All EC2	SSH (your IP only)
80	All EC2	HTTP (Cloudflare → Caddy)
443	All EC2	HTTPS (Cloudflare → Caddy)
4000	All EC2 internal	Backend (not public)
3000	All EC2 internal	Frontend (not public)
5432	All EC2 intra-SG	Postgres replication

Env Variables (backend `.env.prod`)

```
NODE_ENV=production
PORT=4000
DATABASE_URL=postgresql://duitlater:<pass>@postgres:5432/duitlater?schema=public

# Multi-cloud AI
ALIBABA_FUNCTION_COMPUTE_URL=https://<fc>.fcv3.ap-southeast-1.fc.aliyuncs.com/
2023-03-30/proxy/duitlater-fc/penasihat-suggest/
ALIBABA_FUNCTION_COMPUTE_URL_NADI=
ALIBABA_FUNCTION_COMPUTE_KEY=
ANTHROPIC_API_KEY=sk-ant-xxxxx

# TNG (sandbox simulated for hackathon)
TNG_API_BASE=https://sandbox.tngwallet.com.my
TNG_CLIENT_ID=
TNG_CLIENT_SECRET=
TNG_WEBHOOK_SECRET=

# Logging
LOG_LEVEL=info
```

Endpoints

URL	Purpose
`https://duitlater.com/api/v1/health`	Health check (Cloudflare uses this)
`https://duitlater.com/api/v1/penasihat/suggest`	Penasihat AI suggester
`https://duitlater.com/api/v1/nadi/summary`	NADI weekly summary
`https://<fc-url>`	Alibaba FC direct (for testing)
`https://dashboard.cloudflare.com/...`	Cloudflare LB analytics

Useful Logs

```
# Backend app
docker logs duitlater-prod-app --tail 100 --follow

# Postgres
docker logs duitlater-prod-postgres --tail 100 --follow

# Caddy
docker logs duitlater-prod-caddy --tail 100 --follow

# System cron
sudo journalctl -u cron --since "1 hour ago"

# Backup script
tail -f /var/log/duitlater-backup.log

# Mirror script
tail -f /var/log/duitlater-mirror.log
```

Penutup

Setup ni mungkin ambik **4-6 jam first time** untuk Mung implement penuh (provision EC2 + replication + Cloudflare + Alibaba FC + backup). Kalau dah ada experience dengan Postgres replication, lagi cepat (~3 jam).

Demo strategy untuk hackathon judging: 1. Show normal flow — pengguna access `duitlater.com`, traffic to Server 1 2. SSH stop app on Server 1 — dramatic moment 3. Wait ~90s, judges see traffic auto-route ke Server 2 di Cloudflare dashboard 4. Show Postgres replication still working (insert on primary, query replica) 5. Optional: live promote Server 2 to primary for full DB failover demo 6. Highlight: “Multi-cloud — AI workloads are on Alibaba Cloud Function Compute, AWS handles compute + storage”

Ringkasan strategi multi-cloud orchestration:

Layer	Cloud	Component
DNS + LB + WAF	**Cloudflare**	Auto-failover · health monitoring · global edge
Compute (web)	**AWS EC2** × 3	Active-passive HA cluster
Database	**AWS EC2** Postgres	Streaming replication, manual primary promotion
AI inference	**Alibaba Cloud Function Compute**	Qwen LLM (BM-native)
AI fallback	**Anthropic** Claude	When Alibaba 5xx
Object storage	**AWS S3**	Postgres backups (hourly)
Cross-cloud DR	**Alibaba OSS**	S3 mirror (daily)

Sponsor alignment = AWS (Gold) + Alibaba Cloud (Platinum) — both visible, both functionally critical.

Kalau ada bahagian yang tak jelas atau perlu Mung clarify, catatkan dalam team-ledger atau ping di group. Failover playbook printout berguna untuk hari demo — print Bahagian D + Bahagian E sahaja untuk reference cepat.

Authored by: Prime (AI orchestration) **Reviewer:** Mung (Backend Lead) · Kairu (PM gate) **Last updated:** 2026-04-25 **Companion:** `infra/RELEASE.md` (single-EC2 baseline)

Sendiri tak mampu, ramai-ramai boleh.